
Smart Waste Bin

Project Report — Advanced Programming Techniques

Team 10 - Wasted

Vasileios Banakos · Charalampos Papadopoulos · Ioanna Trochatou

Abstract: A Raspberry Pi senses bin usage and gas events, publishes them over MQTT, and the data is stored, served, analysed, predicted, and visualised across a fully containerised stack.

GitHub Repository: [IoannaTroch/Smart_Waste_Bin_Project](https://github.com/IoannaTroch/Smart_Waste_Bin_Project)

Contents

1	Introduction and Objectives	2
2	System Architecture	2
2.1	Why broker-centric?	2
2.2	Identity convention	3
3	Milestone/Technology stack table	3
4	Milestones	3
4.1	M1. Project Foundation	3
4.2	M2. PIR Integration and Local Logging	4
4.3	M3. Modular Pipeline Components	4
4.4	M4. Containerisation	4
4.5	M5. JSON-LD Data Modelling	4
4.6	M6. MQTT Broker, Producer and Consumer	4
4.7	M7. Home Assistant Discovery and Online Status	5
4.8	M8. REST API and AsyncAPI Specification	5
4.9	M9. Virtual Sensors (Rules + Machine Learning)	5
4.10	M10 — Node-RED Low-Code Layer	5
4.11	M11. Dashboards and Analytics	5
5	The Data Model in Detail	6
6	MQTT Topic Scheme	6
7	Challenges, Problems and Lessons Learned	6
8	Conclusions and Future Work	7

1 Introduction and Objectives

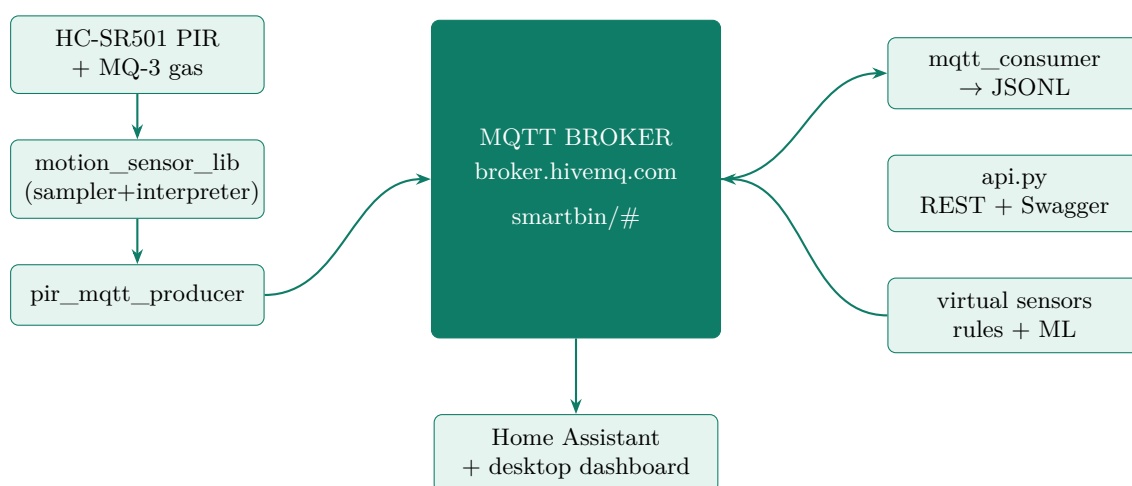
The Wasted project demonstrates a complete IoT pipeline from a physical sensor attached to a microcontroller to high level analytics and interactive dashboards. A Raspberry Pi 5 equipped with an HC-SR501 passive infra-red (known as PIR) motion sensor and an MQ-3 gas sensor acts as the edge node. So, each time someone approaches the bin and opens it, a motion event is generated while the gas sensor continuously monitors for ethanol and volatile fumes. These raw observations travel over the MQTT publish/subscribe system to a containerised backend that:

- persists every event to a log
- serves the data through a formally documented REST API
- derives new “virtual” observations using a rule engine and a machine-learning model
- visualises the live system in Home Assistant, a Node-RED flow, and a custom desktop dashboard
- and finally analyses historical data with six Seaborn charts.

The guiding design principle is loose coupling through a message broker meaning that every component only knows about MQTT topics, never about other components. This makes the system modular, independently testable, and easy to extend , thus having precisely the engineering qualities this course is designed to teach.

2 System Architecture

The system is organised in three tiers: the edge/Raspberry Pi, the broker/public HiveMQ, and the backend/Docker services, plus presentation surfaces ,Home Assistant and the desktop dashboard. As said previously all inter-component traffic flows through the broker, so no service ever connects directly to another.



2.1 Why broker-centric?

As each tier only speaks MQTT, the producer does not need to know whether a consumer, a database, three dashboards, or nothing at all is listening-consuming. As a result, components

can be restarted, moved to different machines, or replaced entirely without touching the rest of the system. This is the publish/subscribe decoupling pattern that underpins real IoT platforms at scale.

2.2 Identity convention

A single logical identity, `bin-01` / `pir-01`, is used everywhere: the edge node, both virtual sensors, the Home Assistant entities, and the JSON-LD model files. This single consistent identity is what allows the rule sensor, the dashboard, and the API to all line up on the same data stream with no translation layer.

3 Milestone/Technology stack table

Layer	Technology	Role
Edge hardware	Raspberry Pi 5, HC-SR501, MQ-3	Sense motion and gas at the bin
GPIO access	gpiozero + rpi-lgpio	Read sensor pins on Pi OS Bookworm
Messaging	MQTT (paho-mqtt), HiveMQ	Decoupled publish/subscribe transport
Persistence	JSON Lines (.jsonl)	Append-only, crash-safe event log
REST API	Flask + Flask-RESTx	OpenAPI/Swagger documented HTTP interface
Machine learning	scikit-learn (RandomForest)	Predict busy/quiet for the next hour
Data model	JSON-LD (schema.org, SOSA/SSN)	Self-describing, machine-readable entities
Low-code	Node-RED	Visual flow for filtering, counting, alerting
Visualisation	Home Assistant, Seaborn, Tkinter	Live status and historical analytics
Orchestration	Docker + Docker Compose	One-command, fully reproducible deployment

4 Milestones

The project was built incrementally across eleven milestones (M1–M11), each adding one distinct capability on top of the previous.

4.1 M1. Project Foundation

The goal was to establish a clean repository layout, dependency strategy, and single source of truth. The repository made separates concerns into `pi_edge_node/` (everything that touches hardware), `src/` (backend services), `models/` (the data model), `home_assistant/`, `node_red/`, and `laptop_dashboard/` as visualization technics. A top-level `requirements.txt` supports local development, meaning that each containerised tier has its own slim requirements file to keep image sizes small.

4.2 M2. PIR Integration and Local Logging

The goal was to prove we can read the PIR sensor and persist clean events.

So file `pir_smoke_test.py` writes directly to the GPIO pin and prints every voltage change — the first proof that the wiring is correct. File `pir_event_logger.py` records debounced motion events to a JSON Lines file, one JSON object per line. JSON Lines was chosen because it is append-only and crash-safe: a half-written last line never corrupts earlier records.

4.3 M3. Modular Pipeline Components

The goal was to use Refactor sensing into small, single-responsibility, reusable parts.

The `motion_sensor_lib` package splits sensing into two classes:

- `PirSampler` -> only reads the raw HIGH/LOW state of the GPIO pin. It knows nothing about events or debouncing.
- `PirInterpreter` -> a pure-Python state machine that converts the noisy raw stream into clean `motion_detected` events using two tunables: `min_high_s` (rejects brief electrical spikes) and `cooldown_s` (rate-limits repeated triggers).

Because the interpreter has no hardware dependency, it can be unit-tested on any laptop. This separation is reused by the logger, the debug tool, and the MQTT producer.

4.4 M4. Containerisation

Here the goal was to bring the entire backend up with one command.

So `docker-compose.yml` defines all services: a one-shot model trainer, the edge producer, the consumer, the API, two virtual sensors, and Home Assistant. Two Dockerfiles produce two images: a backend image (`src/Dockerfile`) shared by the API, consumer, trainer, and virtual sensors, and an edge image (`pi_edge_node/Dockerfile`) for the producer. The producer container is given hardware access via `privileged: true` and a `/dev` bind-mount so it can reach the Pi's GPIO. A named Docker volume, `smartbin_data`, is shared across all services that need the trained model or the event log.

4.5 M5. JSON-LD Data Modelling

Here is needed to describe bins, sensors, and the environment as machine-readable linked data.

The `models/` directory holds JSON-LD documents that reuse established vocabularies: `schema.org` for names, locations, and event timestamps; `SOSA/SSN` for sensors and observations; and a small project namespace (`urn:smartbin:vocab#`) for domain-specific concepts such as usage intensity. A shared `context.jsonld` defines every term once; `wastebin.jsonld`, `sensor.jsonld`, and `environment.jsonld` reference it. The sensor registry deliberately includes the two *virtual* sensors alongside the physical PIR, treating derived observations as first-class entities.

4.6 M6. MQTT Broker, Producer and Consumer

Here happens the transition from local file logging to a networked, decoupled pipeline.

The producer (`pir_mqtt_producer.py`) publishes a family of topics under `smartbin/<bin>/<device>/...`: a plain `motion` state, a `gas` state, a rich JSON-LD `events` message, plus retained `event_count` and `last_motion` values. The consumer (`mqtt_consumer.py`) subscribes to `smartbin/+ /+ /events`,

stamps each message with an ingest timestamp and source topic, and appends it to `data/motion_events.jsonl`. The public HiveMQ broker is used so no broker needs to be operated locally.

4.7 M7. Home Assistant Discovery and Online Status

As a visualization mode, the goal was to make the device self-describing and always observable via Home assistant.

So, on connect, the producer publishes Home Assistant *MQTT discovery* config messages for each entity (motion, gas, online, event count, last motion). It also registers an MQTT **Last Will and Testament**: a retained `online=false` message that the broker delivers automatically if the Pi drops off the network, so dashboards immediately show the node as offline. Discovery means entities appear in Home Assistant with no manual configuration YAML.

4.8 M8. REST API and AsyncAPI Specification

Here we wanted to add a pull-based interface alongside the push-based MQTT one.

So `api.py` (Flask-RESTx) exposes resources for bins, sensors, per-bin events, virtual-sensor readings, and an MQTT bridge, with an auto-generated Swagger UI at the root URL. The MQTT interface is documented separately in `docs/asyncapi.yaml` (AsyncAPI is the event-driven analogue of OpenAPI), so both surfaces are formally specified and explorable.

4.9 M9. Virtual Sensors (Rules + Machine Learning)

The goal was to derive higher-level observations from the raw sensor stream.

Two independent virtual sensors subscribe to the pipeline and publish new derived topics:

- **Rule-based** (`virtual_sensor_rules.py`) counts detections in a rolling time window and maps the count to `idle/low/medium/high`, publishing to `.../usage`.
- **ML-based** (`virtual_sensor_ml.py`) loads a RandomForest trained by `train_model.py` on time features (hour, day of week, weekend flag) and predicts whether the *next* hour will be `busy` or `quiet`, publishing a confidence value alongside.

Both register themselves in Home Assistant and are queryable through the API's `/virtual` endpoint, making AI predictions appear as “just another sensor.”

4.10 M10 — Node-RED Low-Code Layer

The goal was to demonstrate the same logic through a visual, low-code tool.

The `node_red/flows.json` subscribes to `smartbin/+ /+ /motion`, filters for detections, counts them in a sliding window, classifies usage intensity, and raises an alert on `smartbin/<bin>/alert` when activity is high — all without a single line of imperative Python code, complementing the backend services with a graphical pipeline that is easy to inspect and modify.

4.11 M11. Dashboards and Analytics

Finally the goal was to present the system to humans, both live and historically.

Three surfaces were built:

1. A **Home Assistant** dashboard fed by auto-discovery, with colour-coded states, conditional alert banners, and a system overview.

2. A **live desktop dashboard** (`mqtt_gui_consumer.py`, Tkinter + Matplotlib) showing live KPIs, a scrolling event feed, four real-time charts, gas alerts, and online status.
3. An **offline analytics** script (`analyze.py`, Seaborn) that turns the historical JSONL log into six charts: events per hour, daily trend, a day-of-week heatmap, inter-event latency distribution and trend, and per-bin event totals.

Both visual tools include a synthetic-data **demo mode** so they can be showcased without any physical hardware connected.

5 The Data Model in Detail

JSON-LD lets ordinary JSON carry an explicit, shared vocabulary so that machines can interpret it. In our model, a bin is a `schema:Place`-like container that is `smartbin:monitored` by a sensor; a sensor is a `sosa:Sensor smartbin:mounted_on` a bin; and the environment `schema:contains` bins. On the wire, each motion event is a `schema:Event` carrying `startDate`, `madeBySensor`, `hasSimpleResult`, and a `location`. Because every term resolves through the shared `context.jsonld`, the data is self-describing and could be merged with other linked-data sources without any schema negotiation.

6 MQTT Topic Scheme

Topic	Payload	Published by
<code>smartbin/<bin>/<dev>/motion</code>	detected / clear	edge node
<code>smartbin/<bin>/<dev>/gas</code>	detected / clear	edge node
<code>smartbin/<bin>/<dev>/events</code>	rich JSON-LD	edge node
<code>smartbin/<bin>/<dev>/event_ count</code>	integer (retained)	edge node
<code>smartbin/<bin>/<dev>/last_ motion</code>	ISO timestamp	edge node
<code>smartbin/<bin>/<dev>/online</code>	true / false (LWT)	edge node
<code>smartbin/<bin>/usage</code>	JSON usage level	rule sensor
<code>smartbin/<bin>/prediction</code>	JSON busy/quiet	ML sensor
<code>smartbin/<bin>/alert</code>	JSON alert	Node-RED

7 Challenges, Problems and Lessons Learned

- **Consistent identity.** Early in development, the producer, the rule sensor, and Home Assistant disagreed on the bin/device ID. The rule sensor received no data at all and silently did nothing. Standardising on `bin-01/pir-01` everywhere, in environment variables, CLI flags, YAML, and code, fixed an entire class of “no data” bugs.
- **Build context matters.** A Docker image only contains what the Dockerfile `COPYs` from its build context. When the backend image accidentally used the edge node’s Dockerfile, `/app/src` was never copied and the training job failed with “can’t open file.” The lesson: verify what’s inside an image with `docker compose run --rm <svc> ls /app/src` rather than assuming.

- **Caches lie.** A rebuild finishing in seconds is reusing the layer cache. `--build` on `compose up` is not the same as `docker compose build --no-cache`.
- **Schema agreement.** The REST API filtered events by a field the producer did not emit, and returned an empty list every time. The producer and consumer must agree on the exact event schema — a shared reference document (this report) helps keep that contract explicit.

**These are real problems we encountered throughout the whole build of the project

8 Conclusions and Future Work

The project delivers a working, fully containerised IoT system that spans hardware sensing, MQTT messaging, persistent storage, a documented REST API, rule-based and ML-derived virtual sensors, low-code Node-RED flows, and multiple dashboards — all decoupled through a public MQTT broker and deployable with a single command.

In the future we would like to append to the project:

- An ultrasonic sensor for true fill-level detection.
- Retraining the ML model on the real accumulated event log instead of synthetic data, improving prediction quality over time.
- Migrating the JSONL log to an InfluxDB time-series database for faster queries and longer retention.
- Adding OAuth2 authentication to the broker and REST API for production deployments.
- Extending the topic scheme to support multiple bins and locations, enabling city-scale monitoring with minimal code changes.